

Informe técnico: mfshield_mario

Tema de *Super Mario Bros* en la Multi-Function Shield
con la NUCLEO-L476RG, programado en ensamblador ARM

7 de octubre de 2026

1 Resumen

`mfshield_mario` es un programa en ensamblador ARM para la placa NUCLEO-L476RG que toca el tema principal de *Super Mario Bros* con el zumbador de la Multi-Function Shield y muestra en el display de 7 segmentos la frecuencia, en hercios, de la nota que suena. No usa HAL ni interrupciones: escribe directamente en los registros de RCC, GPIOA, GPIOB y TIM2, y repite la canción sin fin.

Dos decisiones explican casi todo el código:

- **El tono lo genera el hardware.** TIM2 produce una onda cuadrada al 50 % en PB3. Cambiar de nota es escribir dos registros (ARR y CCR2), y la CPU queda libre mientras la nota suena.
- **El tiempo lo marca el display.** La duración de cada nota se cuenta en *barridos* de los cuatro dígitos, de unos 4,2 ms cada uno; el refresco del display y el ritmo de la música salen del mismo bucle.

Tabla 1: Datos generales del proyecto.

Dato	Valor
Placa	NUCLEO-L476RG (STM32L476RG, Cortex-M4) con la Multi-Function Shield de Arduino
Lenguaje	Ensamblador GNU, sintaxis unificada Thumb-2 (<code>Src/main.s</code> , 315 líneas)
Reloj	MSI a 4 MHz, el de arranque: el proyecto no define <code>SystemInit</code>
Periféricos	RCC, GPIOA, GPIOB y TIM2 (canal 2 en modo PWM)
Canción	66 pasos (41 notas y 25 silencios); unos 12 s por vuelta
Tamaño	1340 bytes de flash; 28 bytes de RAM más 1,5 KB reservados para pila y heap
Herramientas	GCC arm-none-eabi 14.3 (STM32CubeIDE 2.2.0), STM32CubeProgrammer, VS Code con Cortex-Debug

2 Estructura del proyecto

Todo el comportamiento está en `Src/main.s`; el resto es el andamiaje de arranque, enlazado y compilación que genera STM32CubeIDE (tabla 2).

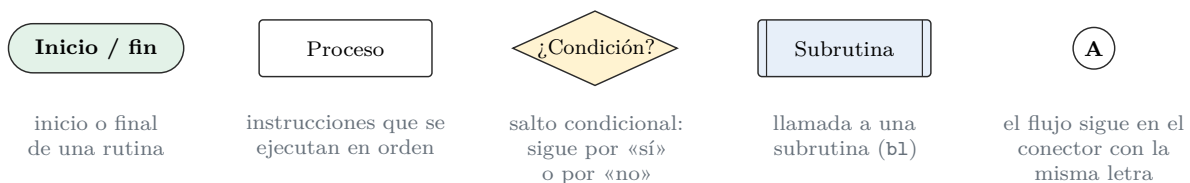
Tabla 2: Archivos del proyecto.

Archivo	Qué es	Papel en el proyecto
Src/main.s	Ensamblador propio	Programa completo: configuración, tono, display y partitura
Startup/ startup_stm32l476rgtx.s	Ensamblador de CubeIDE	Tabla de vectores y <code>Reset_Handler</code> : pila, copia de <code>.data</code> , borrado de <code>.bss</code> y salto a <code>main</code>
STM32L476RGTX_FLASH.ld	Script de enlazado	Mapa de memoria: flash de 1 MB en 0x08000000, RAM de 96 KB en 0x20000000 y SRAM2 de 32 KB
Src/syscalls.c	C de CubeIDE	Llamadas al sistema de newlib (<code>_write</code> , <code>_sbrk</code> , ...); el programa no las usa y el enlazador las descarta
Src/sysmem.c		
Makefile	GNU Make	<code>make</code> compila, <code>make flash</code> graba por SWD y <code>make clean</code> borra <code>build/</code>
.vscode/*.json	Configuración de VS Code	Botones <i>Compilar</i> , <i>Cargar</i> y <i>Limpiar</i> , y depuración por ST-Link con Cortex-Debug
build/	Salida de compilación	Objetos <code>.o</code> , <code>mario.elf</code> (el binario que se graba) y <code>mario.map</code> (mapa de memoria)

Compilar, cargar y depurar. En VS Code, los botones *Compilar*, *Cargar* y *Limpiar* ejecutan `make`, `make flash` y `make clean`. `make flash` graba `build/mario.elf` por SWD con STM32_Programmer_CLI, lo verifica y reinicia la placa. La configuración *Depurar Mario (ST-Link)* compila primero, carga el programa con el servidor GDB de ST-Link y se detiene en `main`. Las rutas de las herramientas en el Makefile y en `.vscode/launch.json` apuntan a `/home/jmaster/st/stm32cubeide_2.2.0/`; en otro equipo hay que ajustarlas.

3 Diagrama de flujo

El programa tiene dos niveles: `main` decide qué nota suena y durante cuánto tiempo (figura 1), y las subrutinas del display convierten ese tiempo en barridos de los cuatro dígitos (figura 2). Los diagramas usan los símbolos habituales:



3.1 Programa principal

La figura 1 sigue la ejecución desde el reset. `Reset_Handler` prepara la pila y la memoria y salta a `main`, que configura relojes, pines y TIM2 una sola vez. Después, el bucle lee la partitura paso a paso: si la duración es 0 la canción terminó, y si la frecuencia es 0 el paso es un silencio. Cada paso termina con una pausa de unos 25 ms; al final de la partitura hay 1,7s de silencio y la canción vuelve a empezar. La tabla 3 recorre las instrucciones de un paso.

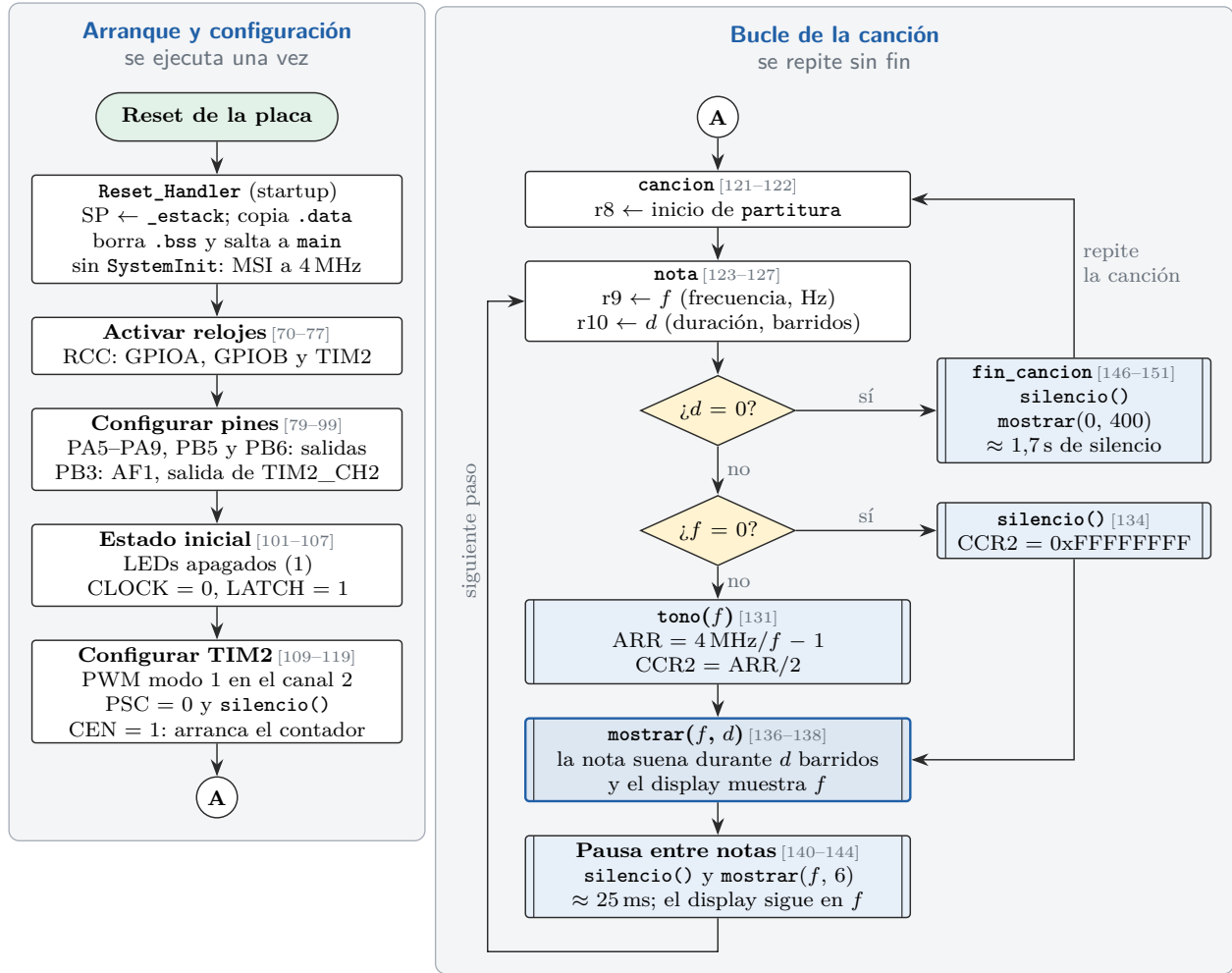


Figura 1: Diagrama de flujo del programa principal. A la izquierda, lo que corre una sola vez tras el reset; a la derecha, el bucle que recorre la partitura. El conector A une las dos columnas. f es la frecuencia del paso ($r9$) y d su duración en barridos ($r10$). La caja resaltada es donde suena cada nota. Entre corchetes, las líneas de `Src/main.s`.

Tabla 3: Un paso de la partitura, instrucción por instrucción.

Líneas	Instrucciones	Qué hacen
124-125	<code>ldrh r9, [r8], #2</code> <code>ldrh r10, [r8], #2</code>	Leen la frecuencia y la duración del paso; $r8$ avanza 4 bytes
126-127	<code>cmp r10, #0; beq fin_cancion</code>	Si la duración es 0, se acabó la partitura
129-130	<code>mov r0, r9</code> <code>cbz r0, nota_silencio</code>	Si la frecuencia es 0, el paso es un silencio
131-134	<code>bl tono (o bl silencio)</code>	Programa TIM2 con la frecuencia, o lo deja callado
136-138	<code>mov r0, r9; mov r1, r10</code> <code>bl mostrar</code>	La nota suena mientras el display muestra la frecuencia durante d barridos
140-143	<code>bl silencio</code> <code>mov r0, r9; movs r1, #6</code> <code>bl mostrar</code>	Corta la nota y espera 6 barridos con la frecuencia todavía en el display
144	<code>b nota</code>	Pasa al siguiente paso

3.2 Subrutinas del display

La figura 2 muestra cómo se dibuja un número en el display y cómo ese dibujo marca el tiempo.

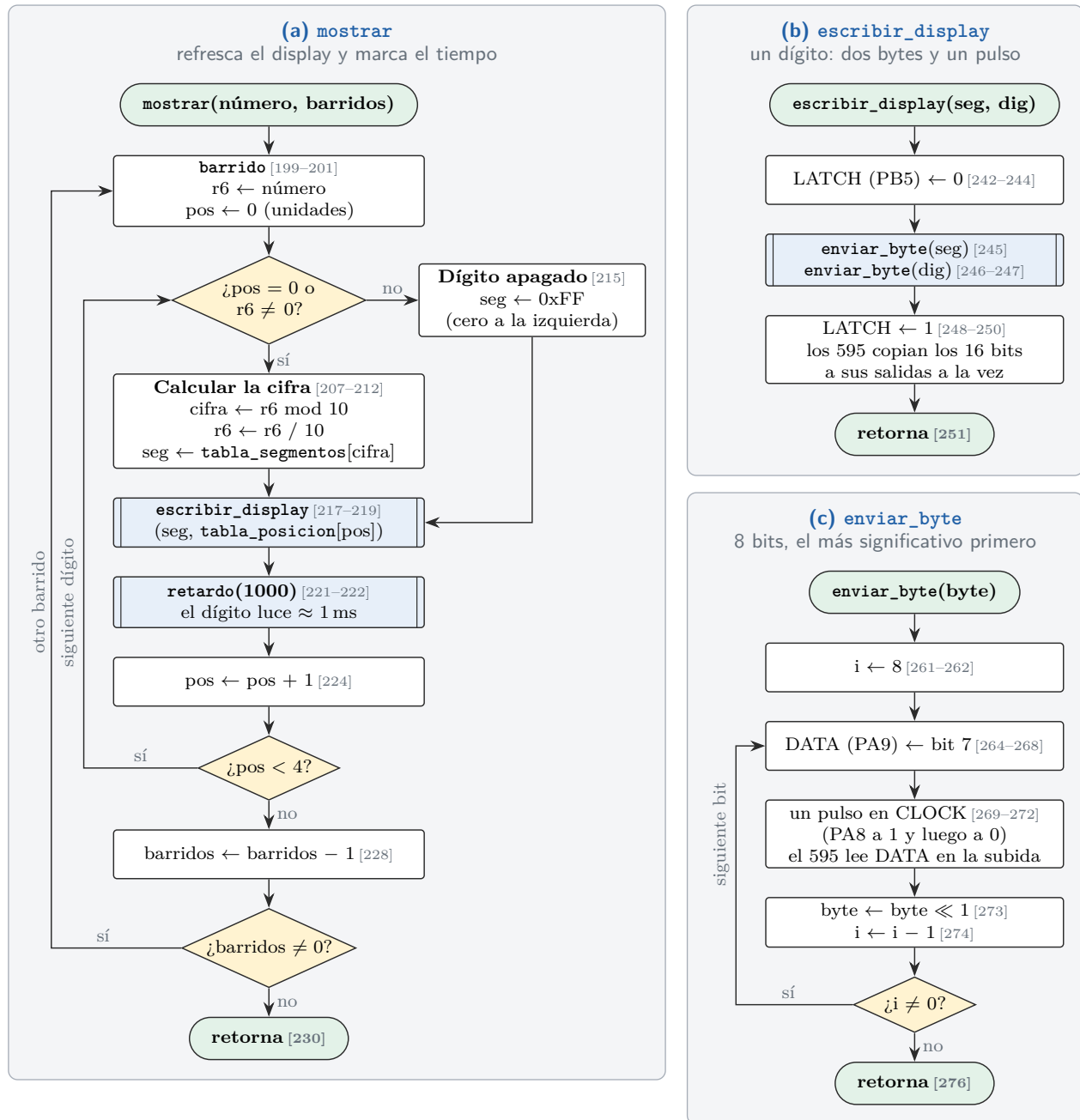


Figura 2: Diagramas de flujo de las subrutinas del display. (a) **mostrar** enciende los cuatro dígitos uno a uno y repite el recorrido tantas veces como barridos se le pidan; (b) **escribir_display** envía los dos bytes de un dígito y da el pulso de LATCH; (c) **enviar_byte** saca un byte bit a bit. pos es la posición del dígito (r4; 0 = unidades) y seg el patrón de segmentos. Entre corchetes, las líneas de **Src/main.s**.

mostrar recorre los cuatro dígitos de las unidades a los miles y repite ese recorrido tantas veces como indique su segundo argumento. En cada dígito calcula la cifra, o lo apaga si es un cero a la izquierda; luego envía 16 bits a los 74HC595 con **escribir_display** y espera cerca de 1 ms con **retardo**. Las unidades siempre se muestran, así que un silencio aparece como «0».

4 Diagrama de conexiones

El programa usa ocho pines del STM32L476RG, y todos llegan a la shield por el conector Arduino de la NUCLEO-L476RG: uno va al zumbador, cuatro a los LEDs y tres a los dos registros de desplazamiento 74HC595 que manejan el display (figura 3 y tabla 4).

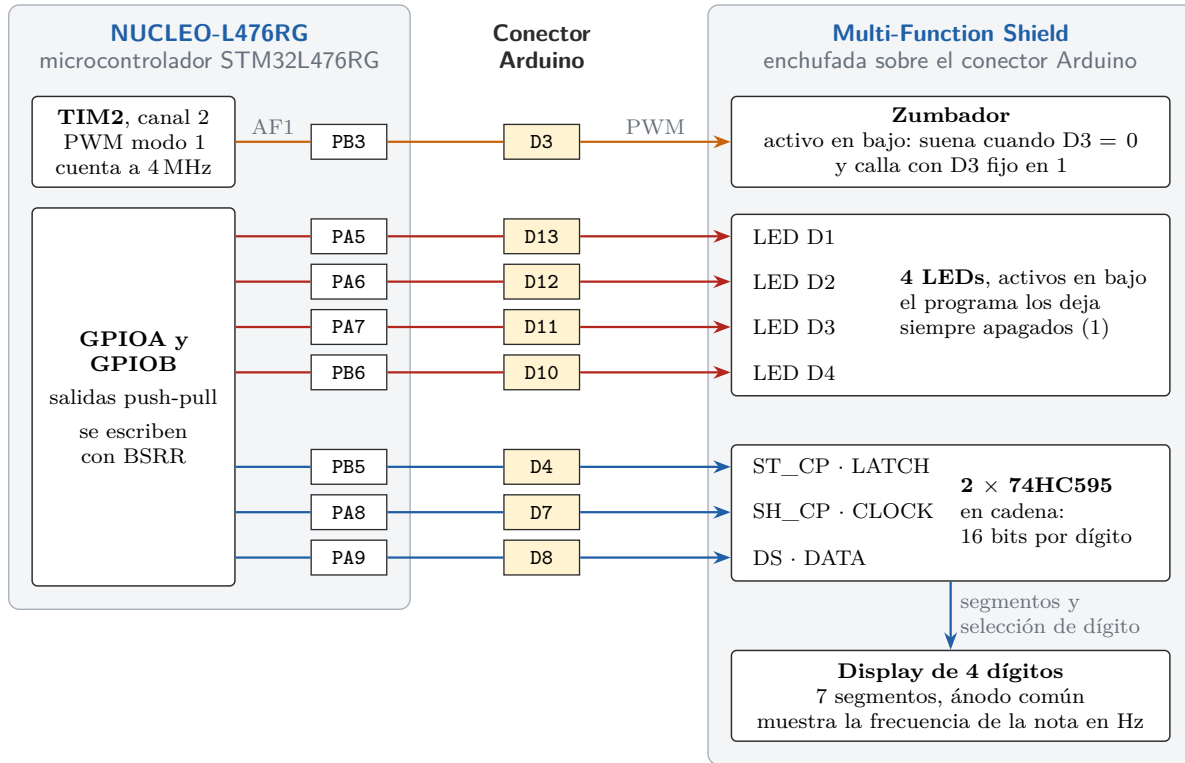


Figura 3: Diagrama de conexiones. Cada señal sale de un periférico del STM32L476RG, pasa por su pin (PB3, PA5...) y llega al conector Arduino de la NUCLEO-L476RG (D3, D13...), donde la shield la recibe. Colores: naranja, zumbador; rojo, LEDs; azul, display. Todas las señales son salidas del microcontrolador.

Tabla 4: Pines que usa el programa.

STM32	Arduino	Modo del pin	Elemento de la shield	Uso en el programa
PB3	D3	AF1 (TIM2_CH2)	Zumbador, activo en bajo	Onda cuadrada de la nota; 1 fijo en los silencios
PA5	D13	Salida	LED D1, activo en bajo	Apagado (1); también enciende LD2 de la NUCLEO
PA6	D12	Salida	LED D2, activo en bajo	Apagado (1)
PA7	D11	Salida	LED D3, activo en bajo	Apagado (1)
PB6	D10	Salida	LED D4, activo en bajo	Apagado (1)
PB5	D4	Salida	74HC595: ST_CP (LATCH)	0 al enviar; al subir, el dígito se muestra
PA8	D7	Salida	74HC595: SH_CP (CLOCK)	Un pulso por bit; el 595 lee DATA al subir
PA9	D8	Salida	74HC595: DS (DATA)	Bit que se envía, el más significativo primero

Los dos 74HC595 comparten CLOCK y LATCH y están en cadena: tras 16 pulsos de reloj, el

primer byte enviado (los segmentos) queda en el segundo registro y el segundo byte (la selección de dígito) en el primero. El flanco de subida de LATCH copia los 16 bits a las salidas a la vez, así que el display nunca muestra un dato a medio enviar.

5 Explicación del código

5.1 Rutinas

Src/main.s tiene una rutina principal y seis subrutinas (tabla 5 y figura 4). Solo **mostrar** y **escribir_display** usan la pila, para guardar los registros que reutilizan; **main** lleva su estado en r8, r9 y r10, que ninguna subrutina modifica.

Tabla 5: Rutinas de Src/main.s.

Rutina	Líneas	Entradas	Qué hace
main	68–151	—	Configura RCC, GPIO y TIM2; recorre la partitura sin fin
tono	160–170	r0 = frecuencia (Hz)	$ARR \leftarrow 4\,000\,000/f - 1$; $CCR2 \leftarrow ARR/2$; UG para aplicarlos ya
silencio	180–186	—	$CCR2 \leftarrow 0xFFFFFFFF$: la salida PWM queda fija en 1
mostrar	195–230	r0 = número (0–9999) r1 = barridos	Enciende los 4 dígitos uno a uno, r1 veces, sin ceros a la izquierda
escribir_display	239–251	r0 = segmentos r1 = selección de dígito	$LATCH \leftarrow 0$, envía los dos bytes y $LATCH \leftarrow 1$
enviar_byte	260–276	r0 = byte	Saca 8 bits por PA9, el más significativo primero, con un pulso en PA8 por bit
retardo	284–287	r0 = número de vueltas	Espera activa: resta 1 hasta llegar a 0

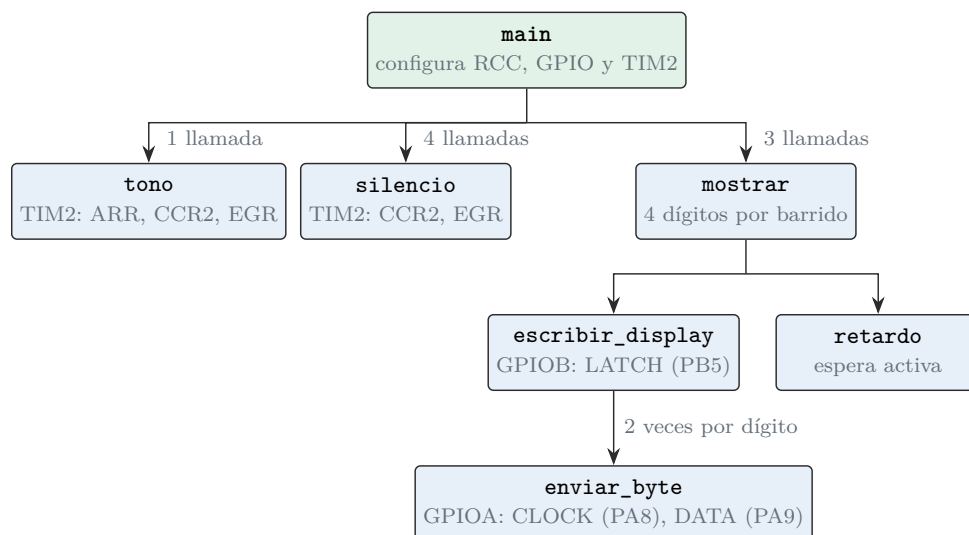


Figura 4: Jerarquía de llamadas: qué rutina llama a cuál y qué periférico escribe cada una. Las notas sobre las flechas de **main** cuentan las instrucciones bl que hay en su código.

5.2 Configuración de periféricos

`main` escribe los registros de la tabla 6. En RCC y en los registros de modo de los GPIO usa secuencias de leer, modificar y escribir (`ldr`, `bic/orr`, `str`), así que solo cambia los bits que necesita; los registros de TIM2 los escribe completos. Para mover los pines usa BSRR: un 1 en los bits 0–15 pone el pin en 1 y un 1 en los bits 16–31 lo pone en 0, sin tocar el resto del puerto.

Tabla 6: Registros que escribe el programa (direcciones del manual de referencia RM0351).

Registro	Dirección	Bits escritos	Efecto	Líneas
RCC_AHB2ENR	0x4002104C	GPIOAEN = GPIOBEN = 1	Reloj de GPIOA y GPIOB	71–74
RCC_APB1ENR1	0x40021058	TIM2EN = 1	Reloj de TIM2	75–77
GPIOA_MODER	0x48000000	MODE5–MODE9 = 01	PA5 a PA9 como salidas	80–86
GPIOB_MODER	0x48000400	MODE5, 6 = 01; MODE3 = 10	PB5, PB6 salidas; PB3 en función alterna	89–95
GPIOB_AFRL	0x48000420	AFSEL3 = 0001	PB3 conectado a TIM2_CH2 (AF1)	96–99
GPIOx_BSRR	base + 0x18	BS (0–15), BR (16–31)	Sube o baja pines sin tocar los demás	102–107, etc.
TIM2_PSC	0x40000028	0	El contador avanza a 4 MHz	111–112
TIM2_CCMR1	0x40000018	OC2M = 110, OC2PE = 1	Canal 2 en PWM modo 1, con precarga de CCR2	113–114
TIM2_CCER	0x40000020	CC2E = 1	Habilita la salida del canal 2	115–116
TIM2_CR1	0x40000000	ARPE = 1, CEN = 1	Precarga de ARR y arranque del contador	118–119
TIM2_ARR	0x4000002C	periodo	Fija la frecuencia de la nota	165
TIM2_CCR2	0x40000038	ancho del pulso	Fija el ciclo de trabajo (o el silencio)	167, 183
TIM2_EGR	0x40000014	UG = 1	Aplica ARR y CCR2 de inmediato	169, 185

5.3 Generación del tono

TIM2 cuenta a 4 MHz ($PSC = 0$) desde 0 hasta ARR y vuelve a empezar. En PWM modo 1, la salida vale 1 mientras el contador es menor que CCR2 y 0 el resto del periodo. Por eso:

$$f_{\text{tono}} = \frac{f_{\text{reloj}}}{(PSC + 1)(ARR + 1)} = \frac{4\,000\,000 \text{ Hz}}{ARR + 1} \quad (1)$$

$$ARR = \left\lfloor \frac{4\,000\,000}{f} \right\rfloor - 1, \quad CCR2 = \left\lfloor \frac{ARR}{2} \right\rfloor \quad (\text{ciclo de trabajo del 50 \%}) \quad (2)$$

`tono` calcula ARR con `udiv` (división entera) y CCR2 con un desplazamiento a la derecha; luego pone UG en TIM2_EGR para que los valores nuevos, que están en precarga, se apliquen de inmediato. `silencio` escribe CCR2 = 0xFFFFFFFF: como ARR siempre es menor, el contador de 32 bits nunca alcanza ese valor, la salida queda fija en 1 y el zumbador, que es activo en bajo, calla. La tabla 7 muestra los valores de cada nota; suponiendo que el MSI oscila exactamente a 4 MHz, el error por redondeo es menor que 0,6 Hz.

5.4 Temporización y partitura

No hay un temporizador para la duración de las notas: el tiempo se cuenta en barridos del display. Un barrido enciende los cuatro dígitos, cada uno durante `retardo(1000)`, y según los comentarios

Tabla 7: Notas de la partitura y valores que carga `tono`.

Constante	Nota	f (Hz)	ARR	CCR2	f real (Hz)	Veces
MI5	Mi5 (E5)	659	6068	3034	659,09	2
SOL5	Sol5 (G5)	784	5101	2550	784,01	5
LA5	La5 (A5)	880	4544	2272	880,09	4
LAS5	La $\sharp$ 5 (A $\sharp$ 5)	932	4290	2145	932,18	2
SI5	Si5 (B5)	988	4047	2023	988,14	4
DO6	Do6 (C6)	1047	3819	1909	1047,12	5
RE6	Re6 (D6)	1175	3403	1701	1175,09	2
MI6	Mi6 (E6)	1319	3031	1515	1319,26	8
FA6	Fa6 (F6)	1397	2862	1431	1397,14	2
SOL6	Sol6 (G6)	1568	2550	1275	1568,01	5
LA6	La6 (A6)	1760	2271	1135	1760,56	2

del código dura unos 4,2 ms. Las duraciones de la tabla 8 son, por tanto, aproximadas: dependen de los ciclos que tarda la CPU en cada vuelta de los bucles.

Tabla 8: Constantes de tiempo (duraciones estimadas con 4,2 ms por barrido).

Constante	Valor	Duración	Para qué sirve
DELAY_DIGITO	1000 vueltas	≈ 1 ms	Tiempo que luce cada dígito
(un barrido)	4 dígitos	$\approx 4,2$ ms	Unidad de tiempo de la canción
T	26 barridos	$\approx 0,11$ s	Nota o silencio básico (2T y 3T para silencios largos)
T3	35 barridos	$\approx 0,15$ s	Notas del tresillo (Sol5, Mi6, Sol6)
PAUSA_NOTAS	6 barridos	≈ 25 ms	Silencio que separa una nota de la siguiente
PAUSA_FINAL	400 barridos	$\approx 1,7$ s	Silencio antes de repetir la canción

La partitura es una lista de pasos de 4 bytes: dos medias palabras (`.hword`) con la frecuencia en Hz (0 = silencio) y la duración en barridos (0 = fin). Así empieza la introducción:

```
.hword MI6, T, MI6, T, SIL, T, MI6, T, SIL, T, DO6, T, MI6, T, SIL, T
```

La introducción tiene 12 pasos y la melodía 27, que se tocan dos veces gracias a `.rept 2`: en total son 66 pasos (41 notas y 25 silencios) que ocupan 268 bytes con el par final 0, 0. Una vuelta completa suma 2878 barridos, unos 12 s: el 72 % se va en notas y silencios, el 14 % en las pausas entre notas y el 14 % restante en la pausa final.

5.5 Display multiplexado

El display es de ánodo común, así que un segmento se enciende con 0. Cada dígito se dibuja con dos bytes: el patrón de segmentos, con los bits en el orden `dp g f e d c b a`, y la selección de posición, que tiene en 1 un solo bit entre el 0 y el 3 (tabla 9). Por ejemplo, `0xC0 = 1100 0000` enciende los segmentos a–f y deja apagados g y el punto: es el «0».

Tabla 9: Tablas del display en `.rodata`: `tabla_segmentos` (arriba) y `tabla_posicion` (abajo).

Cifra	0	1	2	3	4	5	6	7	8	9
Código	C0	F9	A4	B0	99	92	82	F8	80	90

Posición	0	1	2	3
Byte	F8	F4	F2	F1
Dígito	unidades	decenas	centenas	miles

Con unos 240 barridos por segundo, cada dígito luce un cuarto del tiempo y el ojo ve los cuatro encendidos, sin parpadeo. Las posiciones sin cifras quedan apagadas (0xFF): con `□` como dígito apagado, 659 Hz se ve como `□659` y un silencio como `□□□0`. Entre una nota y la siguiente, durante la pausa de 25 ms, el display sigue mostrando la frecuencia de la nota anterior.

6 Observaciones y posibles mejoras

- **Tono y ritmo dependen del reloj.** Si se activa el PLL u otro reloj, hay que cambiar `F_RELOJ` y recalibrar `DELAY_DIGITO`; si no, la música sonará desafinada y a otro tempo. Medir PB3 y PB5 con un osciloscopio confirmaría las duraciones estimadas.
- **main depende de un efecto lateral.** En las líneas 117–119, `main` usa `r2` después de `bl silencio` para escribir `TIM2_CR1`. Funciona porque `silencio` deja `r2 = TIM2_BASE`, pero el AAPCS no garantiza `r2` tras una llamada; recargarlo antes de esa escritura hace el código más robusto.
- **PA5 también enciende LD2.** El LED verde LD2 de la NUCLEO-L476RG está en PA5 (D13) y se enciende en alto: al apagar el LED D1 de la shield (`PA5 = 1`), LD2 queda encendido.
- **PB3 es la línea SWO del depurador.** Usarla como `TIM2_CH2` impide la traza SWO, aunque la depuración por SWD (PA13 y PA14) sigue funcionando.
- **La FPU queda apagada.** Como no hay `SystemInit`, nada la activa en el registro CPACR. Hoy no importa, porque el programa no usa coma flotante; pero código en C con `float`, compilado con `-mfloat-abi=hard`, provocaría un HardFault y se quedaría en el bucle de `Default_Handler`.
- **Hardware sin usar.** Los LEDs D1–D4 se configuran pero siempre están apagados, y los pulsadores S1–S3 (A1–A3) y el potenciómetro (A0) de la shield no se leen. Encender un LED por nota o ajustar el tempo con el potenciómetro serían extensiones naturales.
- **Archivos sobrantes.** `syscalls.c` y `sysmem.c` no aportan código al binario (el enlazador los descarta con `--gc-sections`); pueden quitarse del `Makefile`.

A Código fuente de `Src/main.s`

```

1 /*
2  * main.s
3  *
4  * Multi-Function Shield sobre NUCLEO-L476RG
5  * - Reproduce el tema principal de Super Mario Bros con el zumbador
6  * - El display de 7 segmentos muestra la frecuencia (Hz) de la nota que suena
7  */

```

```

8  * El tono lo genera el TIM2 por hardware (PWM al 50 %):
9  *   ARR = 4 000 000 / frecuencia - 1   (reloj MSI de 4 MHz, sin prescaler)
10 *   CCR2 = ARR / 2
11 *
12 * Conexiones de la shield (Arduino -> Nucleo):
13 *   ZUMBADOR = D3 = PB3 (TIM2_CH2, AF1; activo en bajo)
14 *   LED D1..D4 = D13, D12, D11, D10 = PA5, PA6, PA7, PB6 (activos en bajo)
15 *   74HC595 LATCH = D4 = PB5
16 *   74HC595 CLOCK = D7 = PA8
17 *   74HC595 DATA = D8 = PA9
18 */
19
20 .syntax unified
21
22 .global main
23
24 .equ   RCC_BASE,      0x40021000
25 .equ   RCC_AHB2ENR,   0x4C      // reloj de GPIOA/GPIOB (RM0351, page 251)
26 .equ   RCC_APB1ENR1,  0x58      // reloj de TIM2 (RM0351, page 255)
27
28 .equ   GPIOA_BASE,    0x48000000 // GPIO BASE ADDRESS (RM0351, page 78)
29 .equ   GPIOB_BASE,    0x48000400
30 .equ   GPIO_MODER,    0x00      // GPIO port mode register (RM0351, page 303)
31 .equ   GPIO_BSRR,     0x18      // bits 0-15 ponen el pin en 1, bits 16-31 en 0
32 .equ   GPIO_AFR1,     0x20      // funcion alterna de los pines 0..7 (RM0351, page 307)
33
34 .equ   TIM2_BASE,     0x40000000 // (RM0351, page 1033 en adelante)
35 .equ   TIM_CR1,       0x00
36 .equ   TIM_EGR,       0x14
37 .equ   TIM_CCMR1,     0x18
38 .equ   TIM_CCER,      0x20
39 .equ   TIM_PSC,       0x28
40 .equ   TIM_ARR,       0x2C
41 .equ   TIM_CCR2,      0x38
42
43 .equ   F_RELOJ,        4000000   // MSI a 4 MHz (valor de arranque)
44 .equ   DELAY_DIGITO,   1000     // ~1 ms encendido por dígito del display
45 .equ   PAUSA_NOTAS,    6         // silencio corto entre notas (barridos)
46 .equ   PAUSA_FINAL,    400       // silencio antes de repetir la canción
47
48 // Duracion en barridos del display (cada barrido dura ~4.2 ms)
49 .equ   T,              26        // tiempo basico ~0.11 s
50 .equ   T3,             35        // nota de tresillo ~0.15 s
51
52 // Frecuencias de las notas (Hz)
53 .equ   MI5,            659
54 .equ   SOL5,           784
55 .equ   LA5,            880
56 .equ   LAS5,          932       // La sostenido
57 .equ   SI5,           988
58 .equ   DO6,          1047
59 .equ   RE6,          1175
60 .equ   MI6,          1319
61 .equ   FA6,          1397
62 .equ   SOL6,         1568
63 .equ   LA6,          1760
64 .equ   SIL,           0         // silencio
65
66 .section .text.main
67 .type   main,%function
68 main:
69
70 // ACTIVAR RELOJ DE GPIOA, GPIOB Y TIM2
71     ldr r2, =RCC_BASE
72     ldr r1, [r2, #RCC_AHB2ENR]
73     orr r1, #0x3             // GPIOA y GPIOB
74     str r1, [r2, #RCC_AHB2ENR]
75     ldr r1, [r2, #RCC_APB1ENR1]
76     orr r1, #0x1             // TIM2

```

```

77     str r1, [r2, #RCC_APB1ENR1]
78
79 // PA5..PA9 COMO SALIDA (01): bits 10..19 de MODER
80     ldr r2, =GPIOA_BASE
81     ldr r1, [r2, #GPIO_MODER]
82     ldr r3, =(0x3FF << 10)
83     bic r1, r3
84     ldr r3, =(0x155 << 10)
85     orr r1, r3
86     str r1, [r2, #GPIO_MODER]
87
88 // PB5, PB6 COMO SALIDA (01) Y PB3 COMO FUNCION ALTERNA (10)
89     ldr r2, =GPIOB_BASE
90     ldr r1, [r2, #GPIO_MODER]
91     bic r1, #(0xF << 10)
92     orr r1, #(0x5 << 10)
93     bic r1, #(0x3 << 6)
94     orr r1, #(0x2 << 6)
95     str r1, [r2, #GPIO_MODER]
96     ldr r1, [r2, #GPIO_AFRL] // PB3 -> AF1 (TIM2_CH2)
97     bic r1, #(0xF << 12)
98     orr r1, #(0x1 << 12)
99     str r1, [r2, #GPIO_AFRL]
100
101 // ESTADO INICIAL: LEDs apagados (1), CLOCK en 0, LATCH en 1
102     ldr r2, =GPIOA_BASE
103     ldr r1, =((0x7 << 5) | (1 << (8+16)))
104     str r1, [r2, #GPIO_BSRR]
105     ldr r2, =GPIOB_BASE
106     ldr r1, =((1 << 6) | (1 << 5))
107     str r1, [r2, #GPIO_BSRR]
108
109 // TIM2 EN MODO PWM 1 POR EL CANAL 2
110     ldr r2, =TIM2_BASE
111     movs r1, #0
112     str r1, [r2, #TIM_PSC] // sin prescaler: cuenta a 4 MHz
113     ldr r1, =((6 << 12) | (1 << 11))
114     str r1, [r2, #TIM_CCMR1] // OC2M = 110 (PWM 1), OC2PE = 1
115     movs r1, #(1 << 4)
116     str r1, [r2, #TIM_CCER] // CC2E = 1: habilita la salida del canal 2
117     bl silencio
118     movs r1, #((1 << 7) | 1)
119     str r1, [r2, #TIM_CR1] // ARPE = 1, CEN = 1: arranca el timer
120
121 cancion:
122     ldr r8, =partitura
123 nota:
124     ldrh r9, [r8], #2 // r9 = frecuencia (Hz), 0 = silencio
125     ldrh r10, [r8], #2 // r10 = duracion (barridos), 0 = fin
126     cmp r10, #0
127     beq fin_cancion
128
129     mov r0, r9
130     cbz r0, nota_silencio
131     bl tono
132     b nota_sonar
133 nota_silencio:
134     bl silencio
135 nota_sonar:
136     mov r0, r9
137     mov r1, r10
138     bl mostrar // muestra la frecuencia mientras suena
139
140     bl silencio // separa una nota de la siguiente
141     mov r0, r9
142     movs r1, #PAUSA_NOTAS
143     bl mostrar
144     b nota
145

```

```

146 fin_cancion:
147     bl silencio
148     movs r0, #0
149     ldr r1, =PAUSA_FINAL
150     bl mostrar
151     b cancion
152 .size    main, .-main
153
154
155 /*
156  * tono: r0 = frecuencia en Hz
157  * Programa el periodo del TIM2 y deja el ciclo de trabajo en 50 %.
158  */
159 .type    tono,%function
160 tono:
161     ldr r1, =F_RELOJ
162     udiv r1, r1, r0
163     subs r1, #1                // r1 = ARR = 4 MHz / f - 1
164     ldr r2, =TIM2_BASE
165     str r1, [r2, #TIM_ARR]
166     lsr r1, r1, #1
167     str r1, [r2, #TIM_CCR2]    // mitad del periodo en alto, mitad en bajo
168     movs r1, #1
169     str r1, [r2, #TIM_EGR]    // UG: carga ARR y CCR2 de inmediato
170     bx lr
171 .size    tono, .-tono
172
173
174 /*
175  * silencio: deja PB3 siempre en 1 (zumbador apagado).
176  * En PWM 1 la salida esta en 1 mientras CNT < CCR2, y CCR2 = 0xFFFFFFFF
177  * nunca se alcanza.
178  */
179 .type    silencio,%function
180 silencio:
181     ldr r2, =TIM2_BASE
182     mov r1, #0xFFFFFFFF
183     str r1, [r2, #TIM_CCR2]
184     movs r1, #1
185     str r1, [r2, #TIM_EGR]
186     bx lr
187 .size    silencio, .-silencio
188
189
190 /*
191  * mostrar: r0 = numero (0..9999), r1 = numero de barridos del display
192  * Multiplexa los 4 digitos; los ceros a la izquierda quedan apagados.
193  */
194 .type    mostrar,%function
195 mostrar:
196     push {r4-r7, lr}
197     mov r5, r0
198     mov r7, r1
199 barrido:
200     mov r6, r5                // r6 = numero que se va descomponiendo
201     movs r4, #0                // r4 = posicion (0 = unidades ... 3 = miles)
202 digito:
203     cbz r4, digito_numero     // las unidades siempre se muestran
204     cbz r6, digito_apagado    // cero a la izquierda: ya no quedan cifras
205 digito_numero:
206
207     movs r1, #10
208     udiv r2, r6, r1            // r2 = r6 / 10
209     mls r3, r2, r1, r6        // r3 = r6 - r2*10 = digito actual
210     mov r6, r2
211     ldr r0, =tabla_segmentos
212     ldrb r0, [r0, r3]         // r0 = patron de segmentos del digito
213     b digito_enviar
214 digito_apagado:

```

```

215     movs r0, #0xFF           // todos los segmentos apagados
216 digito_enviar:
217     ldr r1, =tabla_posicion
218     ldrb r1, [r1, r4]         // r1 = seleccion del digito en el display
219     bl escribir_display
220
221     ldr r0, =DELAY_DIGITO
222     bl retardo
223
224     adds r4, #1
225     cmp r4, #4
226     blt digito
227
228     subs r7, #1
229     bne barrido
230     pop {r4-r7, pc}
231 .size mostrar, .-mostrar
232
233
234 /*
235  * escribir_display: r0 = segmentos, r1 = seleccion de digito
236  * Envia los dos bytes a los 74HC595 y luego da el pulso de LATCH.
237  */
238 .type escribir_display,%function
239 escribir_display:
240     push {r4, lr}
241     mov r4, r1
242     ldr r2, =GPIOB_BASE
243     mov r1, #(1 << (5+16))    // LATCH en 0
244     str r1, [r2, #GPIO_BSRR]
245     bl enviar_byte            // primero los segmentos
246     mov r0, r4
247     bl enviar_byte            // luego la seleccion de digito
248     ldr r2, =GPIOB_BASE
249     mov r1, #(1 << 5)         // LATCH en 1: los 595 muestran los datos
250     str r1, [r2, #GPIO_BSRR]
251     pop {r4, pc}
252 .size escribir_display, .-escribir_display
253
254
255 /*
256  * enviar_byte: r0 = byte, se envia el bit mas significativo primero
257  * DATA = PA9, CLOCK = PA8 (el 595 lee el dato en el flanco de subida)
258  */
259 .type enviar_byte,%function
260 enviar_byte:
261     ldr r2, =GPIOA_BASE
262     movs r3, #8
263 bit:
264     tst r0, #0x80
265     ite ne
266     movne r1, #(1 << 9)        // DATA = 1
267     moveq r1, #(1 << (9+16))   // DATA = 0
268     str r1, [r2, #GPIO_BSRR]
269     mov r1, #(1 << 8)          // CLOCK = 1
270     str r1, [r2, #GPIO_BSRR]
271     mov r1, #(1 << (8+16))     // CLOCK = 0
272     str r1, [r2, #GPIO_BSRR]
273     lsl r0, r0, #1
274     subs r3, #1
275     bne bit
276     bx lr
277 .size enviar_byte, .-enviar_byte
278
279
280 /*
281  * retardo: r0 = numero de vueltas
282  */
283 .type retardo,%function

```

```

284 retardo:
285     subs r0, #1
286     bne retardo
287     bx lr
288 .size   retardo, .-retardo
289
290
291 .section .rodata
292 // Display de anodo comun: segmento encendido = 0 (bits: dp g f e d c b a)
293 tabla_segmentos:
294     .byte 0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90    // 0..9
295 // Seleccion de digito: indice 0 = unidades (derecha) ... 3 = miles (izquierda)
296 tabla_posicion:
297     .byte 0xF8, 0xF4, 0xF2, 0xF1
298
299 // Tema de Super Mario Bros: pares (frecuencia Hz, duracion); duracion 0 = fin
300 .balign 2
301 partitura:
302     // Introduccion
303     .hword MI6, T, MI6, T, SIL, T, MI6, T, SIL, T, D06, T, MI6, T, SIL, T
304     .hword SOL6, T, SIL, 3*T, SOL5, T, SIL, 3*T
305
306     // Melodia (se toca dos veces)
307     .rept 2
308     .hword D06, T, SIL, 2*T, SOL5, T, SIL, 2*T, MI5, T, SIL, 2*T
309     .hword LA5, T, SIL, T, SI5, T, SIL, T, LAS5, T, LA5, T, SIL, T
310     .hword SOL5, T3, MI6, T3, SOL6, T3
311     .hword LA6, T, SIL, T, FA6, T, SOL6, T, SIL, T, MI6, T, SIL, T
312     .hword D06, T, RE6, T, SI5, T, SIL, 2*T
313     .endr
314
315     .hword 0, 0

```